AURORA.

TÀI LIỆU THIẾT KẾ VÀ BẢO VỆ ĐỒ ÁN

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

JAVA 21 • JAVAFX • REST / SSE • MVC

Bài tập lớn Lập trình nâng cao • UET.CS2043

Kiến trúc • OOP • Concurrency • Observer • Kiểm thử

Bản tài liệu học và tập bảo vệ — cần kiểm thử đầy đủ trước khi nộp

Năm 2026 • AURORA Online Auction System

1. Tổng quan đồ án

AURORA là hệ thống đấu giá trực tuyến đa client. Website và ứng dụng JavaFX cùng gọi REST API đến một Java Server. Người dùng được phân vai Bidder, Seller và Admin. Máy chủ là nguồn dữ liệu giá duy nhất.

Mục tiêu theo đề môn học

- OOP: abstract Entity, User, Item; kế thừa, đóng gói, đa hình, trừu tượng.
- MVC: JavaFX + FXML, Server Controller → Model/Service → Repository (DAO).
- Concurrency: tránh lost update, giá rollback và hai người thắng cùng lúc.
- Realtime: Observer / SSE phát sự kiện đến mọi client theo dõi.
- Chất lượng: Maven, JUnit 5, GitHub Actions, xử lý ngoại lệ.

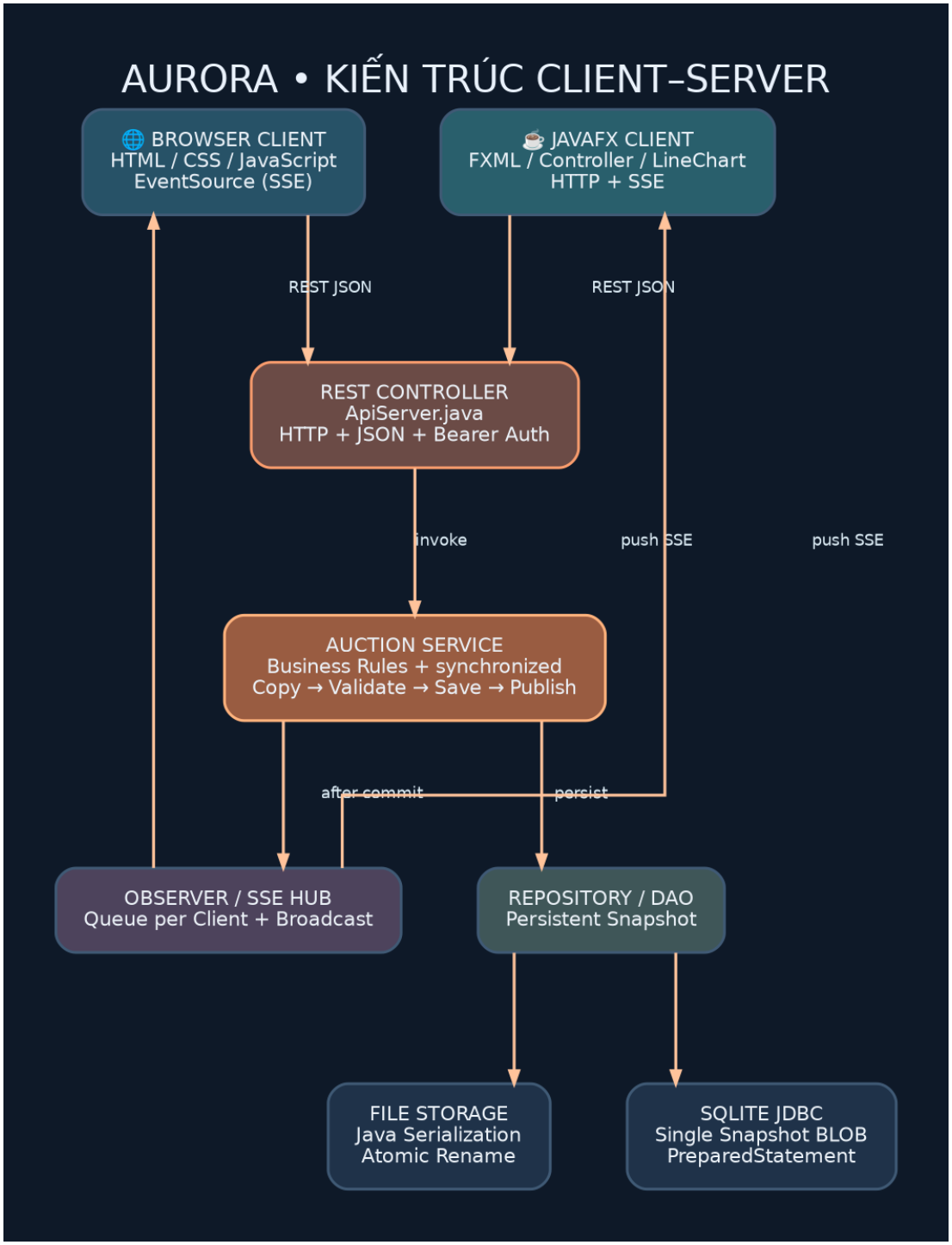
Nhóm tính năng có trong mã nguồn

Nhóm	Chức năng
Tài khoản	Đăng ký, đăng nhập, phân vai Bidder/Seller/Admin
Đấu giá	Tạo/sửa/xóa sản phẩm, đặt giá, lưu history, auto close
Nâng cao	Auto-bid, anti-sniping, biểu đồ giá cập nhật theo event
Hệ thống	REST/JSON, SSE, File Serialization, tùy chọn SQLite JDBC

Lưu ý: Server và HTTP core đã được biên dịch, chạy kiểm thử. JavaFX/Maven/CI cần được chạy xác nhận trên máy có đầy đủ thư viện và Internet.

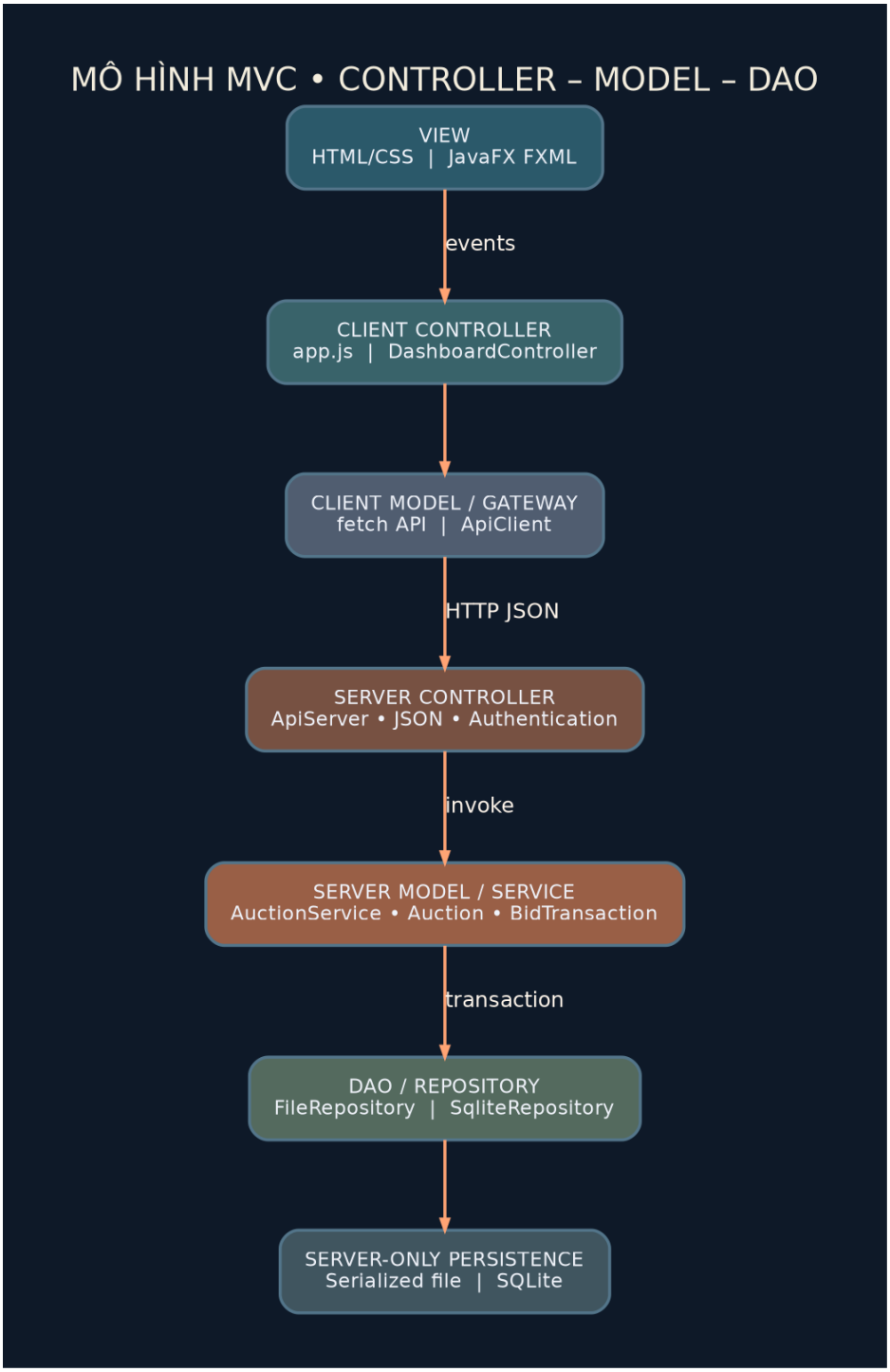
2. Kiến trúc Client–Server

Hai giao diện độc lập chia sẻ một server. Mọi giá đấu và lưu trữ do server quản lý.



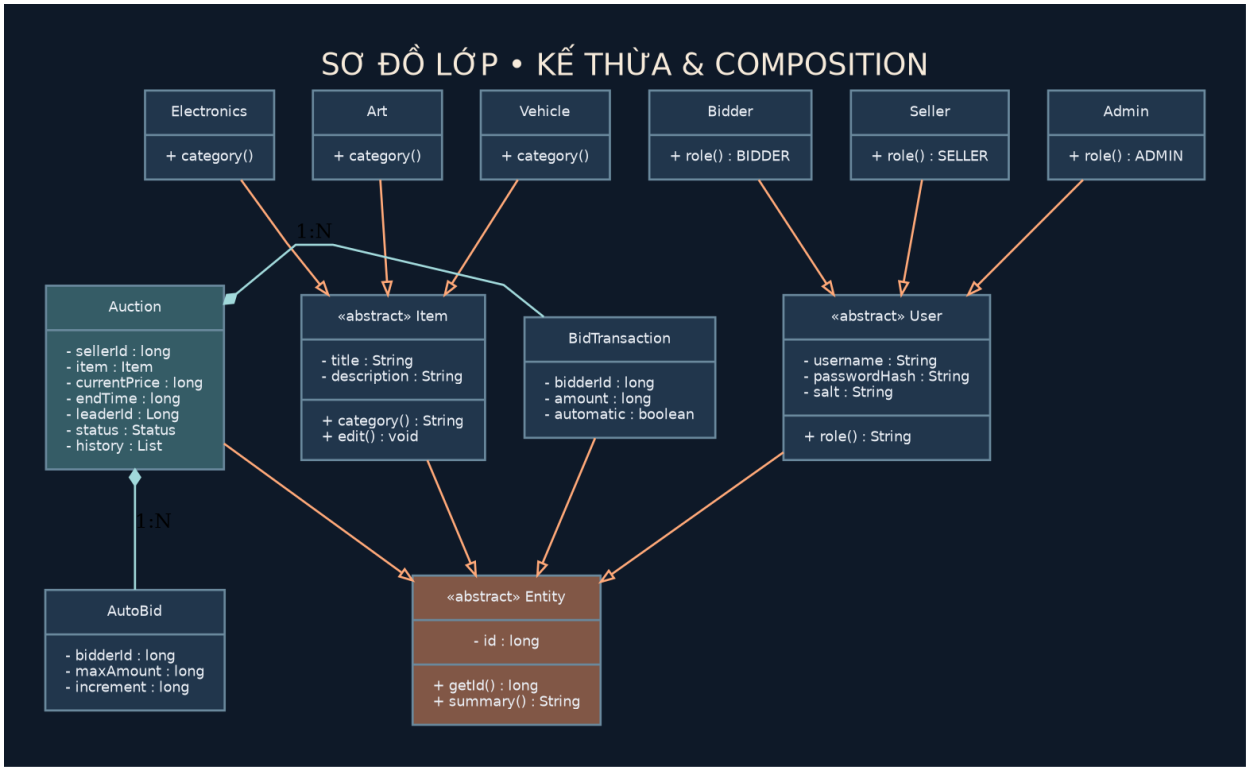
3. Kiến trúc logic MVC / DAO

Sơ đồ cho thấy tách riêng hiển thị, điều khiển, nghiệp vụ và dữ liệu.



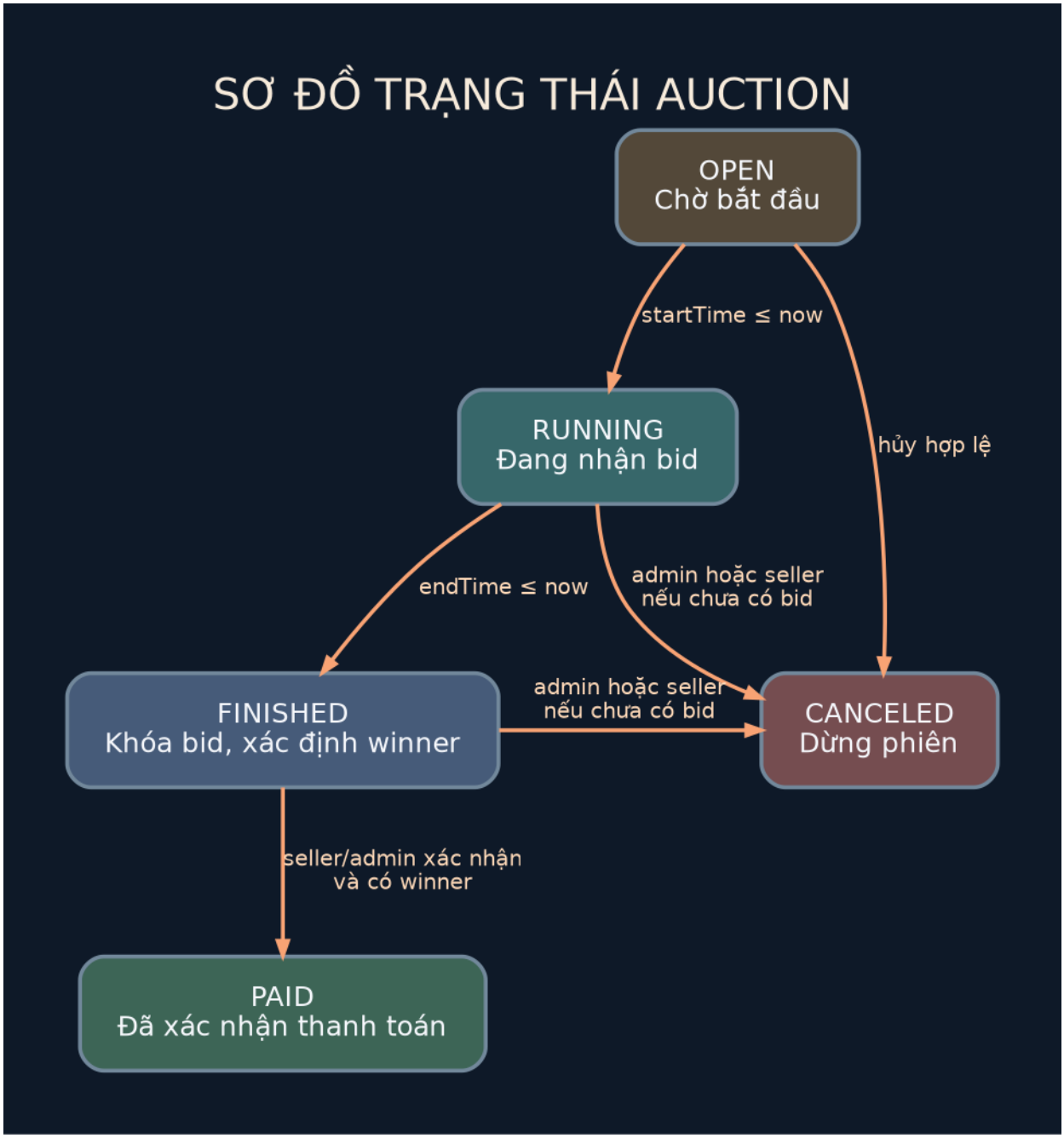
4. Sơ đồ lớp OOP

Kế thừa từ Entity/User/Item, composition trong Auction, hỗ trợ Factory Method.



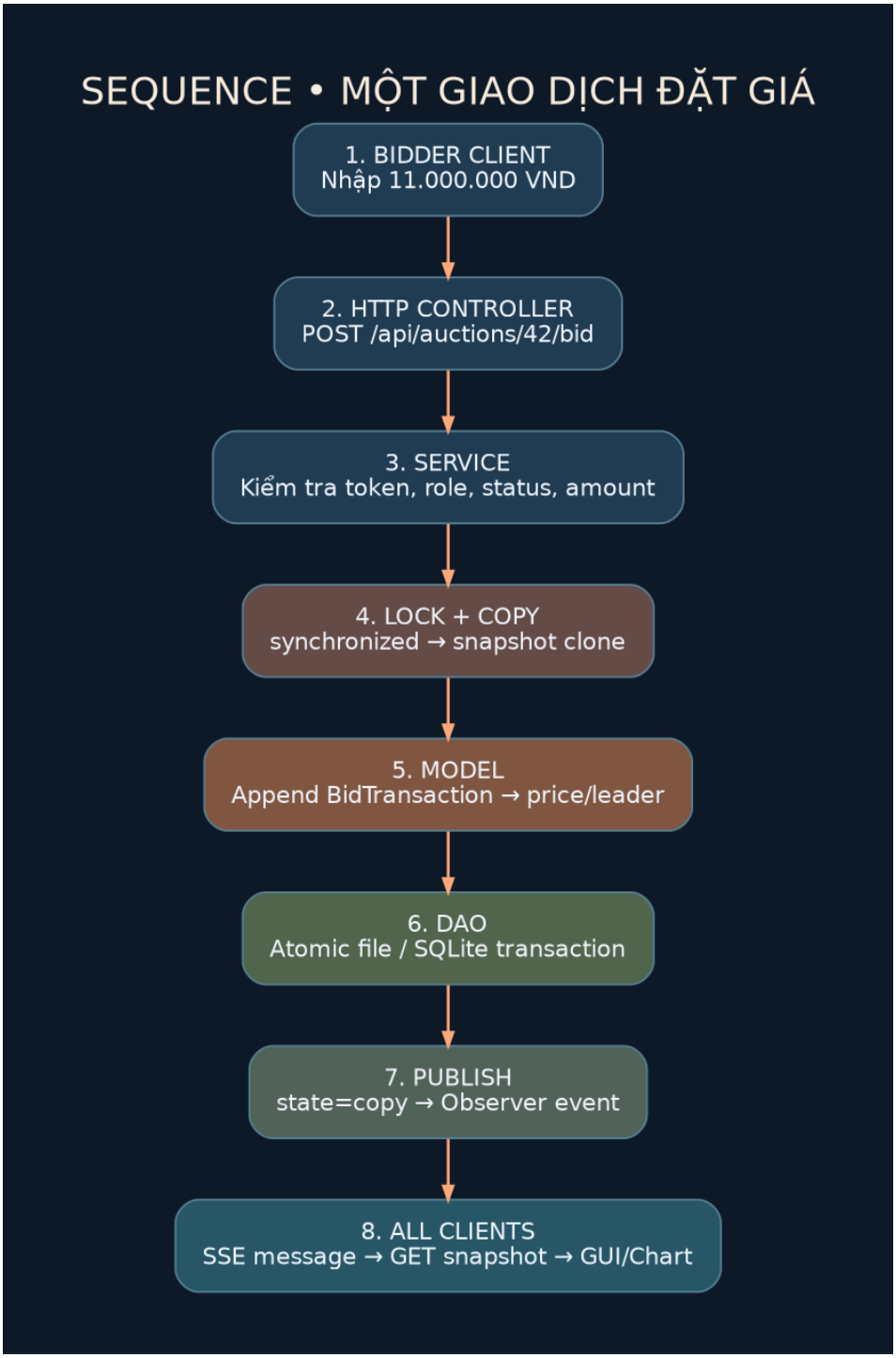
5. Máy trạng thái phiên đấu giá

OPEN → RUNNING → FINISHED → PAID; CANCELED dùng khi được phép hủy.



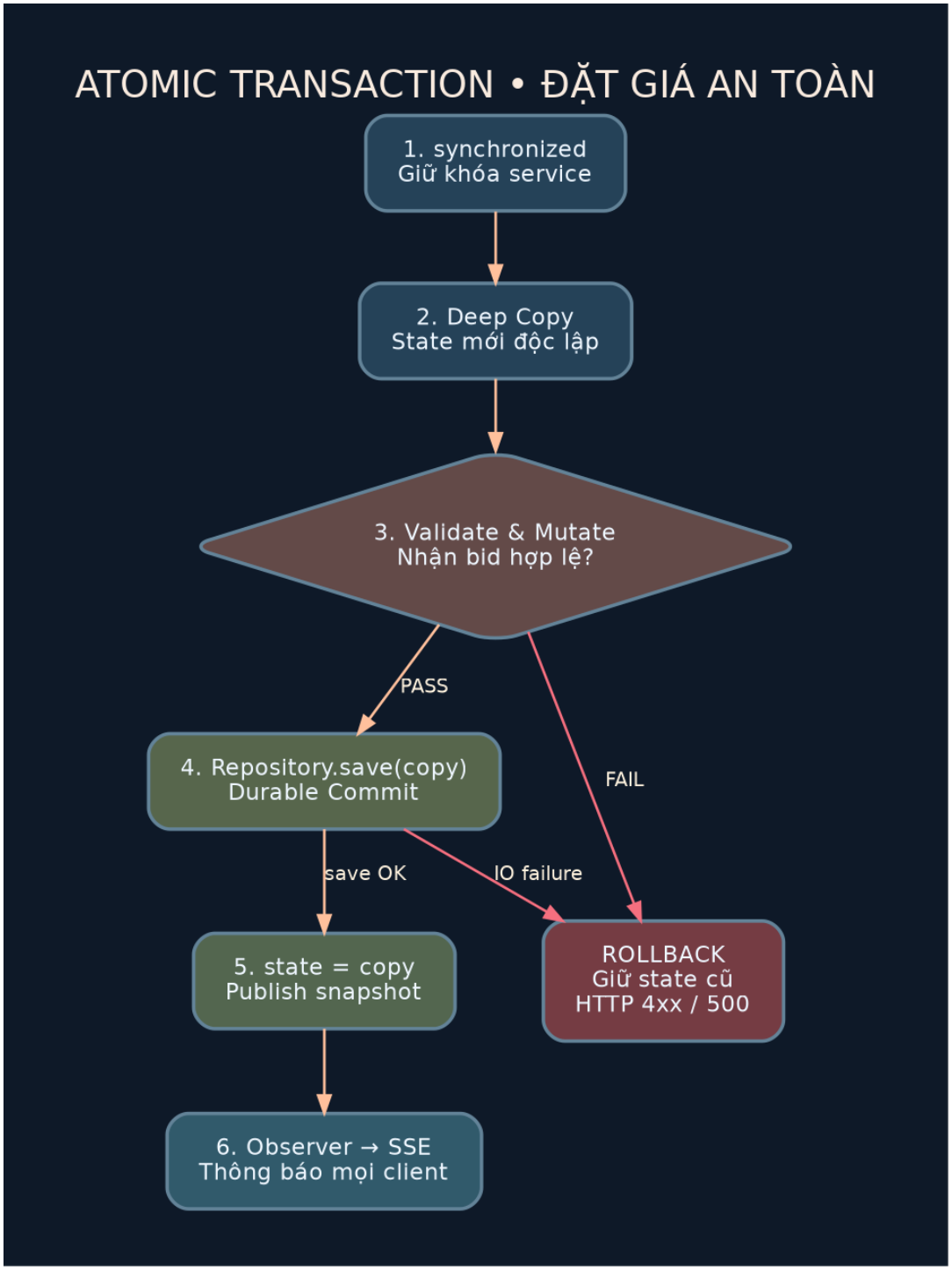
6. Sơ đồ tuần tự một lượt bid

Client gửi request; server khóa giao dịch, lưu thành công mới công bố sự kiện.



7. Sơ đồ logic đảm bảo atomicity

Clone → validate → persist → publish bảo toàn giá khi kiểm tra hoặc ghi dữ liệu thất bại.



8. Thuật toán cạnh tranh giá đồng thời

Giả sử giá đang là 10.000.000 VND. Bidder A và B cùng đặt 11.000.000 VND. Nếu cả hai thread cùng đọc giá cũ rồi ghi đè thì dẫn đến lost update hoặc báo hai người thắng.

Quy trình trong AuctionService.bid()

- 1. Java khóa một service instance qua synchronized.
- 2. Thread A lấy state hiện tại, sao chép sang state tạm.
- 3. A hợp lệ, ghi một BidTransaction, currentPrice=11 triệu, leaderId=A.
- 4. Repository lưu state mới, sau đó service mới thay state chính thức và gửi Observer event.
- 5. B vào sau và đọc currentPrice=11 triệu nên bid bằng 11 triệu bị từ chối (HTTP 422).

Bất biến (invariants) cần bảo vệ

- currentPrice của các bid đã chấp nhận phải tăng nghiêm ngặt.
- Tại một thời điểm Auction chỉ có duy nhất một leaderId.
- Tuyệt đối không notify client giá thành công trước khi dữ liệu đã lưu.
- Một giao dịch lỗi phải để state chính thức không thay đổi.

Giới hạn kiến trúc

Khóa synchronized chỉ áp dụng trong một JVM, không phải khóa phân tán. Khi mở rộng nhiều server cần DB row locking/optimistic locking; ở bản môn học ưu tiên một server đảm bảo tính đúng.

9. Auto-bidding • Anti-sniping • Realtime

Auto-bidding

Người dùng đặt maxAmount và increment. Server lưu một AutoBid và trong mỗi vòng tạo PriorityQueue xếp theo max giảm dần, timestamp đăng ký tăng dần. Các rule của người đang dẫn đầu được loại khỏi lượt phản hồi; chỉ chấp nhận giá mới nếu currentPrice+increment ≤ maxAmount. Mỗi bid tự động được ghi lịch sử với automatic=true.

Ví dụ: A đăng ký max=8.000, bước=1.000; B trả giá 3.000; server tự đặt 4.000 cho A. Khi B đặt 9.000, rule A không còn đủ ngân sách và B dẫn đầu. Thuật toán bản mẫu chặn ở 500 bước để tránh vòng lặp quá dài.

Anti-sniping

Trong 30 giây cuối, một bid hợp lệ sẽ kéo dài thời hạn đến ít nhất currentServerTime+60 giây. Ví dụ end=20:00:00, bid=19:59:50 ⇒ end tối thiểu 20:00:50. Dùng Math.max để không bao giờ rút ngắn thời gian.

Realtime Observer

Observer trong AuctionService phát sự kiện chỉ sau khi commit. SSE giữ một HTTP connection; trình duyệt nhận qua EventSource, JavaFX đọc stream nền. Sau event, client GET snapshot mới để hiển thị giá và vẽ biểu đồ. Đếm ngược mỗi giây chỉ là thay đổi UI local, không phải polling server.

Bảo mật ở mức học phần

Hash mật khẩu PBKDF2-HMAC-SHA256 với salt ngẫu nhiên; Bearer token thời hạn 24 giờ, RBAC tại server. Chưa có HTTPS/rate-limit/production hardening, không chạy công khai trên Internet.

10. Kịch bản demo thực tế (8–12 phút)

Thời gian	Thao tác demo	Ý nghĩa
0:00–2:00	Giới thiệu sơ đồ kiến trúc và cây kế thừa.	Giải thích Client–Server, OOP, MVC.
2:00–4:00	Seller đăng nhập, thêm/sửa sản phẩm.	CRUD, role, validation.
4:00–6:00	2 Bidder ở 2 trình duyệt cùng mở phiên và tăng giá.	Realtime push, lịch sử, kiểm tra giá.
6:00–7:30	Bật auto-bid; một bidder khác trả giá.	Max/increment, PriorityQueue.
7:30–9:00	Xem biến động giá trên chart; mở thêm JavaFX.	Hai client dùng chung REST server.
9:00–12:00	Chạy smoke/JUnit test cạnh tranh, demo anti-sniping.	Không lost update; timer trên server.

Các lệnh cần thuộc

Server: mvn -pl server exec:java

JavaFX: mvn -pl client-javafx javafx:run

JUnit: mvn test

Core smoke khi thiếu Maven: bash scripts/test-core.sh

HTTP end-to-end test: python scripts/test-http.py

11. Câu hỏi phản biện thường gặp

1. MVC khác OOP như thế nào?

MVC chia trách nhiệm kiến trúc giữa Model/View/Controller; OOP tổ chức các đối tượng dùng kế thừa, đóng gói, trừu tượng, đa hình.

2. Vì sao client không tự tính giá?

Người dùng có thể sửa client. Giá chính thức phải kiểm tra ở server dùng dữ liệu nhất quán.

3. synchronized có đủ cho hai server không?

Không; synchronized chỉ khóa một JVM. Multi-instance cần khóa DB hoặc giao dịch phân tán.

4. Tại sao volatile không đủ?

Volatile cung cấp visibility nhưng không atomic cho chuỗi read–check–write.

5. Nếu hai bidder cùng đưa một giá?

Chỉ một giao dịch giữ khóa trước được chấp nhận; giao dịch sau gặp giá không lớn hơn hiện tại.

6. Vì sao clone state trước khi ghi?

Nếu validation hoặc repository.save lỗi, state đang phục vụ không bị thay đổi.

7. Observer khác polling ra sao?

Observer phát event chủ động sau commit; polling gửi GET theo chu kỳ dù không có thay đổi.

8. SSE và WebSocket có giống nhau không?

Không. SSE là push một chiều server→client trên HTTP, đủ cho thông báo bid; gửi bid dùng POST REST.

9. Thời gian lấy từ đâu?

Dựa vào Clock của server; client clock chỉ dùng hiển thị.

10. Làm thế nào chọn winner?

Auction.leaderId cập nhật sau bid thành công, giữ duy nhất một người dẫn đầu khi FINISHED.

11. Tại sao dùng Factory Method?

Các concrete creator override createItem để tạo Electronics/Art/Vehicle, tách logic tạo đối tượng khỏi nghiệp vụ.

12. Có thể sửa sản phẩm sau khi có bid không?

Không, bản này chỉ cho phép sửa trước khi có bid để tránh đổi thông tin sản phẩm sau khi người dùng quyết định giá.

13. Auto-bid có thể vượt max không?

Không; mỗi candidate phải thỏa currentPrice+increment ≤ maxAmount.

14. Tại sao phải anti-sniping?

Tránh việc bidder đặt giá vào những giây cuối khiến người khác không có cơ hội phản hồi.

15. Dữ liệu có lưu password gốc không?

Không: PBKDF2-HMAC-SHA256 + salt; chỉ lưu passwordHash/salt.

16. Cần cải tiến gì nếu deploy Internet?

HTTPS, rate-limit, auditing, normalized DB, monitoring, backups, production authentication/session store.

12. Tình trạng kiểm thử và phạm vi trung thực

Đã xác nhận trong môi trường xây dựng

- Java 21: javac biên dịch server thành công.
- scripts/test-core.sh: pass đăng nhập/phân quyền/đặt giá/concurrent/auto/JSON.
- scripts/test-http.py: pass HTTP register/login, seller create, giá không hợp lệ, hai request cùng giá, auto-bid và SSE push.

Chưa xác nhận trong môi trường xây dựng

- Maven, JUnit 5 suite, GitHub Actions: không cài Maven và không thể tải dependencies.
- JavaFX client: đã viết FXML/controller nhưng chưa compile/run giao diện vì thiếu JavaFX runtime.
- SQLite JDBC: có mã nguồn nhưng chưa chạy thực do thiếu sqlite-jdbc jar.
- Web UI thao tác trực tiếp trên trình duyệt: đã có mã nguồn và phục vụ HTML, nhưng môi trường không cho browser automation truy cập localhost.

Lưu ý theo đề bài

Đề môn học yêu cầu commit GitHub đều đặn và các thành viên hiểu mọi phần mã nguồn. Không giả lập lịch sử commit; nhóm phải thật sự học code, thực hiện review chéo và chạy kiểm thử của chính mình trước khi trình bày.

Tài liệu chi tiết hơn: Xem README.md và docs/*.md trong ZIP. Mọi sơ đồ có phiên bản .dot để chỉnh sửa và .svg/.png để đưa vào slide.